Guia de Comandos e Métodos de Vetores (Arrays) em JavaScript

Em JavaScript, **vetores** (ou *arrays*) possuem dezenas de métodos nativos para inserção, remoção, busca, transformação e ordenação de dados. Abaixo está a relação dos principais comandos categorizados pelo objetivo prático.

1. Inserir e Remover Elementos

Método	O que faz	Modifica o original?	Retorno
push(...)	Insere um ou mais itens no **fim**	Sim	Novo tamanho (length) do array
unshift(...)	Insere um ou mais itens no **início**	Sim	Novo tamanho (length) do array
pop()	Remove e entrega o **último** item	Sim	O elemento removido
shift()	Remove e entrega o **primeiro** item	Sim	O elemento removido
splice(inicio, qtd, ...itens)	Remove, substitui ou insere em **qualquer índice**	Sim	Array contendo os itens removidos

Exemplos Práticos:

```
const fila = ["Ana", "Bruno"];

// Adicionando elementos
fila.push("Carlos");      // ["Ana", "Bruno", "Carlos"]
fila.unshift("Zara");     // ["Zara", "Ana", "Bruno", "Carlos"]

// Removendo elementos
const removidoFim = fila.pop();    // "Carlos" (fila: ["Zara", "Ana", "Bruno"])
const removidoInicio = fila.shift(); // "Zara" (fila: ["Ana", "Bruno"])

// Splice: a partir do índice 1, remove 1 item e adiciona "Beto"
```

fila.splice(1, 1, "Beto"); // fila agora é: ["Ana", "Beto"]

2. Localização e Busca

Método	O que faz	Retorno
includes(valor)	Checa se o elemento existe no vetor	true ou false
indexOf(valor)	Busca a 1ª ocorrência do valor	O índice numérico (ou -1 se não achar)
lastIndexOf(valor)	Busca a última ocorrência do valor	O índice numérico (ou -1 se não achar)
find(callback)	Busca o 1º elemento que atende à regra lógica	O próprio elemento (ou undefined)
findIndex(callback)	Busca a posição do 1º elemento que atende à regra	O índice numérico (ou -1)

Exemplos Práticos:

```
const usuarios = [
  { id: 1, nome: "Alice", ativo: false },
  { id: 2, nome: "Bob", ativo: true },
  { id: 3, nome: "Carla", ativo: true }
];

// Checagem simples de valor primitivo
const frutas = ["uva", "maçã", "laranja"];
console.log(frutas.includes("maçã")); // true
console.log(frutas.indexOf("abacaxi")); // -1

// Busca com condição personalizada (find / findIndex)
const usuarioAtivo = usuarios.find(u => u.ativo === true);
console.log(usuarioAtivo); // { id: 2, nome: "Bob", ativo: true }

const posicaoBob = usuarios.findIndex(u => u.nome === "Bob");
console.log(posicaoBob); // 1
```

3. Transformação e Filtragem (Programação Funcional)

Esses métodos **não modificam o array original**; eles geram novos dados.

Método	Objetivo	Retorno
map(fn)	Transforma cada item aplicando uma regra	Um **novo array** com o mesmo tamanho
filter(fn)	Filtra apenas os itens que retornarem true	Um **novo array** (tamanho menor ou igual)
reduce(fn, inicial)	Acumula todos os itens em um único valor final	O valor final acumulado (número, objeto, etc.)
flat(nivel)	"Achata" sub-vetores internos em um nível único	Um novo array linearizado

Exemplos Práticos:

```
const precos = [10, 20, 30, 40];

// 1. map: aplicar 10% de desconto
const precosComDesconto = precos.map(p => p * 0.9);
// [9, 18, 27, 36]

// 2. filter: manter apenas valores maiores que 20
const caros = precos.filter(p => p > 20);
// [30, 40]

// 3. reduce: calcular o total da compra
const total = precos.reduce((acumulador, precoAtual) => acumulador + precoAtual, 0);
// 100
```

4. Divisão, Junção e Cópia

Método	O que faz	Modifica original?	Retorno
slice(inicio, fim)	Extrai uma fatia do array (índice final não incluso)	Não	Novo array fatiado
concat(outroArray)	Junta dois ou mais vetores	Não	Novo array combinado
join(separador)	Converte todos os elementos em uma única String	Não	String com os itens unidos
[...array] (Spread)	Cria uma cópia superficial (*shallow copy*)	Não	Novo array idêntico

Exemplos Práticos:

```
const letras = ["A", "B", "C", "D", "E"];

// Extrair elementos do índice 1 até antes do índice 4
const fatia = letras.slice(1, 4); // ["B", "C", "D"]

// Criar texto a partir da lista
const texto = letras.join(" - "); // "A - B - C - D - E"

// Combinando vetores com Spread
const lista1 = [1, 2];
const lista2 = [3, 4];
const combinada = [...lista1, ...lista2, 5]; // [1, 2, 3, 4, 5]
```

5. Ordenação e Inversão

Atenção pedagógica: Os métodos tradicionais sort() e reverse() alteram o vetor original diretamente. As versões modernas (toSorted e toReversed) geram um novo vetor preservando o original.

Método Clássico (Mutável)	Versão Moderna ES2023 (Imutável)	O que faz
sort(fn)	**toSorted(fn)**	Ordena os elementos
reverse()	**toReversed()**	Inverte a ordem dos itens

Exemplos Práticos:

```
const numeros = [40, 100, 1, 5, 25];

// ⚠️ Cuidado: numeros.sort() sem função compara como texto ("100" vem antes de "25")
// Para números, use sempre a função de comparação: (a, b) => a - b
numeros.sort((a, b) => a - b);
console.log(numeros); // [1, 5, 25, 40, 100] (ordem crescente)

// Para ordem decrescente:
numeros.sort((a, b) => b - a);
console.log(numeros); // [100, 40, 25, 5, 1]
```

6. Verificações Lógicas em Coleções

Método	Regra	Retorno
every(fn)	Verifica se **TODOS** os elementos passam no teste	true ou false
some(fn)	Verifica se **PELO MENOS UM** elemento passa no teste	true ou false

Exemplos Práticos:

```
const notas = [7.5, 8.0, 6.5, 9.0];

const todosAprovados = notas.every(nota => nota >= 6.0);
console.log(todosAprovados); // true (todas as notas são >= 6)

const temNotaDez = notas.some(nota => nota === 10);
console.log(temNotaDez); // false (nenhuma nota é exatamente 10)
```

Dica de Ouro para Alunos: Mutabilidade

- **Métodos que alteram o array original (Mutáveis):**
  push, pop, shift, unshift, splice, sort, reverse.
- **Métodos que deixam o original intacto (Imutáveis / Seguros):**
  map, filter, slice, concat, reduce, toSorted, toReversed.